I want to design one flagship, production-grade ML Engineering project for my resume. This project should be realistic enough that a fintech company could actually build something similar internally, but it must also be achievable by one developer within approximately 1-1.5 months.

Important constraints:

- This is an ML Engineering/MLOps project, not a full-stack product.
- The focus should be on production ML, deployment, monitoring, automation, and infrastructure.
- Do not keep suggesting generic Kaggle-style projects or multiple unrelated projects.
- We are building one system with a clear architecture.
- Throughout this discussion, think like a Senior ML Engineer or ML Architect reviewing the design.

Project Idea

The project is an **Adaptive Financial Risk Intelligence Engine**.

This is **NOT** a fraud detection project.

This is **NOT** a simple anomaly detection model.

This is **NOT** a credit score prediction project.

The goal is to build a production-ready ML service that continuously evaluates the risk of financial transactions.

For every incoming transaction, the system should:

1. Calculate a transaction risk score.
2. Predict whether the transaction is anomalous or not.
3. Explain WHY the transaction was considered risky.
4. Log predictions and metadata.
5. Monitor model performance.
6. Detect data drift.
7. Trigger retraining when necessary.
8. Version datasets and models.
9. Serve predictions through production APIs.

The project should demonstrate how a real ML system is built, deployed, monitored, and maintained.

Core ML Components

Model 1 — Transaction Risk Engine

Input examples:

- Transaction amount
- Merchant
- Customer history
- Time
- Location (if available)
- Device information (if available)
- Historical spending features
- Engineered behavioral features

Output:

- Risk score (0–100)
- Risk level (Low / Medium / High)
- Anomalous or Normal

This should be the primary ML model.

Explainability Engine

The project should not simply output:

"Fraud"

or

"Anomaly"

Instead it should explain the prediction.

For example:

- Transaction amount is much higher than customer's historical average.
- Merchant has never been seen before.
- Spending behavior differs significantly from historical patterns.
- Transaction occurred at an unusual time.
- Customer's recent behavior changed rapidly.

Use proper explainability techniques (such as SHAP where appropriate) instead of hardcoded reasons.

The explanation should resemble what an analyst at a financial institution would actually see.

MLOps Pipeline

The project must strongly emphasize production ML engineering.

Include:

- Data validation
- Feature engineering pipeline
- Experiment tracking
- Model registry
- Model serving
- Prediction logging
- Monitoring
- Data drift detection
- Model drift detection (if feasible)
- Automated retraining
- Versioning
- CI/CD
- Infrastructure as Code

The prediction model itself is only one part of the project.

Dataset Strategy

The project must use **real, publicly available financial datasets** that resemble production banking systems. Avoid toy datasets whenever possible.

The primary dataset should be:

- **Berka / PKDD'99 Financial Dataset (Czech Bank)** — This will serve as the core relational database because it contains multiple linked tables such as clients, accounts, transactions, loans, cards, and districts. It will be the foundation for feature engineering and risk modeling.

The project may be extended with the following datasets if they provide clear architectural value:

- **IEEE-CIS Fraud Detection** — To enrich the platform with transaction identity, device, browser, and network-related features for anomaly detection.
- **Transactions Fraud Dataset (computingvictor)** — To add customer, card, and transaction relationships and improve behavioral feature engineering.

Do **not** merge datasets arbitrarily. Every additional dataset must solve a specific business or technical problem and fit naturally into the architecture.

The project should be designed so that new data sources can be integrated later through the ingestion pipeline without requiring major architectural changes.

The initial version (Version 1) should be built primarily on the **Berka dataset**, with additional datasets introduced only if they significantly improve the ML pipeline or demonstrate additional MLOps capabilities.

Technologies to Include

I specifically want this project to demonstrate these skills:

- Python
- Scikit-learn
- XGBoost or LightGBM etc
- Airflow
- DVC
- MLflow
- FastAPI
- REST APIs
- Evidently AI
- Terraform
- Kubernetes
- AWS
- GitHub Actions
- Redis
- Docker
- PostgreSQL (if needed and if can be used try to use it)

Only include a technology if it has a real purpose in the architecture.

Do not force technologies unnecessarily.

Project Goals

The project should demonstrate:

- Production ML architecture
- MLOps best practices
- Realistic deployment
- Monitoring and observability
- Explainable AI
- Automation
- Scalability
- Clean software engineering

This should feel like something built by an ML Engineer rather than a Data Scientist working in a notebook.

What I Want from You

Act as a Senior ML Engineer mentoring another ML Engineer.

Challenge my design decisions when necessary.

Do not agree with everything automatically.

Help me make architectural decisions similar to those made in real fintech companies.

Prioritize realism over complexity.

Whenever we introduce a new component, explain:

1. Why it exists.
2. What business problem it solves.
3. Whether it is actually used in production.
4. Whether it is worth adding for resume value.
5. Whether it fits the project or is unnecessary complexity.

Help me build one exceptional project instead of several average ones.

The first thing I want to do is design the complete system architecture before writing any code.

Throughout the project, prioritize architectural quality, production readiness, ML model perdition accuracy and reliability and MLOps depth over adding more ML models or datasets.